

Checklist

Resilience Automation

1. Define Service Level Objectives (SLOs)

- Identify the most critical services.
- Define appropriate Service Level Indicators (SLIs), such as:
 - Availability
 - Error rate
 - Request success rate
 - Latency
- Set an SLO target for each service.
- Define the measurement window, such as a rolling 30-day period.
- Calculate the corresponding error budget.
- Define what happens when the error budget is nearly exhausted.
- Establish deployment-freeze criteria where appropriate.
- Use different SLO targets for services with different business criticality.
- Store SLO definitions as version-controlled configuration.
- Consider a vendor-neutral specification such as OpenSLO.
- Generate Prometheus SLO rules using a tool such as Sloth.
- Publish SLO status through Grafana dashboards.
- Display:
 - Current SLO compliance
 - Error-budget remaining
 - Error-budget burn rate
 - Remaining allowable downtime
- Configure alerts for SLO violations and excessive burn rates.

2. Automate Backup and Restore

- Automate Kubernetes backups instead of relying on manual procedures.
- Back up Kubernetes resources, including:
 - CRDs
 - Secrets

- ConfigMaps
- Persistent volumes
- Configure CSI volume snapshots where applicable.
- Store backups in durable object storage such as S3 or GCS.
- Define backup retention periods.
- Enable cross-region backup replication where DR requires it.
- Schedule recurring backups.
- Support on-demand backups when needed.
- Define a restore procedure.
- Automate restoration of Kubernetes resources.
- Automate restoration of persistent volumes.
- Validate application health after restoration.
- Check logs after restoration.
- Verify restored data integrity.
- Run restore tests in an isolated test cluster.
- Schedule regular restore validation, such as weekly.
- Run functional tests against restored environments.
- Run performance tests where appropriate.
- Provide self-service backup configuration for development teams.
- Allow resources to opt in to backups through labels or configuration.

3. Define Recovery Objectives

- Define an RTO for every important service.
- Confirm the RTO represents the maximum acceptable restoration time.
- Define an RPO for every important service.
- Confirm the RPO represents the maximum acceptable amount of data loss.
- Adjust backup frequency to meet RPO requirements.
- Adjust recovery automation to meet RTO requirements.
- Assign different RTO/RPO targets based on business impact.

4. Establish Chaos Engineering

- Identify meaningful failure scenarios.
- Define the scope of each chaos experiment.
- Define the target workload.
- Define experiment duration.
- Define experiment scheduling.
- Establish expected system behavior before running the experiment.
- Establish abort conditions and safety boundaries.

- Use controlled failure injection rather than uncontrolled testing.
- Test pod failures.
- Test network latency.
- Test packet loss.
- Test CPU stress.
- Test memory stress.
- Test service dependency failures where appropriate.
- Use Chaos Mesh for Kubernetes-native experiments where suitable.
- Schedule experiments predictably.
- Communicate experiment schedules to affected teams.
- Prefer business hours when engineers are available to respond.
- Avoid peak traffic unless peak-load resilience is the explicit objective.
- Run more aggressive experiments during dedicated DR drills or low-traffic periods.

5. Monitor Chaos Experiments

- Monitor Prometheus metrics during experiments.
- Monitor Grafana dashboards.
- Review application logs.
- Track CPU and memory errors.
- Track latency percentiles.
- Track pod restart counts.
- Track request success rates.
- Track database connection-pool behavior.
- Track SLO burn-rate alerts.
- Track recovery-failure alerts.
- Correlate technical failures with business metrics.
- Record users impacted where possible.
- Record revenue or other business impact where relevant.
- Review experiment results after completion.
- Identify resilience gaps.
- Create remediation work.
- Retest after remediation.

6. Test Managed Cloud Services

- Test slow-query scenarios.
- Test connection-limit exhaustion.
- Test managed database failover procedures.

- Test cross-region latency.
- Verify application behavior when managed dependencies degrade.
- Validate graceful degradation paths.
- Verify retry and backoff behavior.
- Test circuit breakers under realistic conditions.
- Test managed-service recovery procedures before an actual outage occurs.

7. Test Kubernetes Workloads

- Kill individual pods.
- Kill multiple pods simultaneously.
- Inject sustained pod failures.
- Induce network failures.
- Introduce latency and packet loss.
- Stress CPU resources.
- Stress memory resources.
- Verify ReplicaSets replace failed pods.
- Measure recovery time.
- Monitor restart counts.
- Check whether application availability remains within its SLO.

8. Prepare Disaster Recovery

- Identify which systems must survive catastrophic failures.
- Define recovery priorities.
- Determine whether multi-region architecture is required.
- Configure a primary region.
- Configure a secondary or standby region where required.
- Replicate databases across regions.
- Replicate object-storage backups across regions.
- Replicate cache/data services where necessary.
- Maintain a standby Kubernetes environment where appropriate.
- Configure DNS health checks.
- Configure a documented failover mechanism.
- Define replica-promotion procedures.
- Define application-restoration procedures.
- Ensure monitoring operates during failover.
- Verify the architecture can meet required RTO targets.
- Verify replication can meet required RPO targets.
- Run regular DR drills.

- Measure actual recovery time during drills.
- Compare actual recovery performance against RTO/RPO targets.
- Update procedures based on test results.

9. Chaos Mesh Exercise Validation

- Install the Chaos Mesh Helm repository.
- Update the Helm repository index.
- Install Chaos Mesh in the `chaos-mesh` namespace.
- Confirm `chaos-controller-manager` is running.
- Confirm `chaos-dashboard` is running.
- Create the `chaos-testing` namespace.
- Deploy a three-replica `nginx:alpine` application named `demo-app`.
- Confirm all three pods are running.
- Apply `chaos-mesh-pod-failure.yaml`.
- Confirm Chaos Mesh accepts the first experiment.
- Observe pod events in real time.
- Monitor pod restart counts.
- Review namespace events by timestamp.
- Measure how quickly replacement pods are created.
- Check `chaos_mesh_` metrics in Prometheus/Grafana.
- Compare chaos metrics with observed pod churn.
- Delete experiment resources after testing.
- Remove the `chaos-testing` namespace.

10. Continuous Resilience Improvement

- Treat resilience testing as an ongoing engineering practice.
- Keep SLO definitions version-controlled.
- Review error-budget consumption regularly.
- Test restores regularly rather than assuming backups work.
- Run scheduled chaos experiments.
- Run scheduled disaster-recovery drills.
- Capture metrics, logs, and observations from every test.
- Document discovered failure modes.
- Remediate weaknesses found during testing.
- Repeat experiments after fixes.
- Practice recovery procedures before real incidents occur.
- Promote continuous learning and improvement across teams.

